Save Point

Database Schema

AlotaNotes Quiz App · Prepared for Logan Young, Save Point Limited · 27 October 2026

Single local SQLite database. `lib/storage/db.dart` holds the full DDL as a list of statements (`schemaStatements`), applied by `sqflite`'s `onCreate` on a fresh install. **Schema version 2** (`schemaVersion`); older databases are upgraded in place by `migrateSchema()` (see below).

`PRAGMA foreign_keys = ON` is set in `onOpen`, after any upgrade, so every `ON DELETE CASCADE` below is enforced. It's deliberately not set in `onConfigure`: the v1 to v2 migration rebuilds tables and must run with foreign keys off.

All timestamps are `TEXT`, UTC ISO-8601, always computed in Dart (never a SQL default).

FSRS columns

`questions` and `problems` share the same five FSRS columns (`_srsColumns`), read and written only through `srsCardToRow()` / `srsCardFromRow()`:

`fsrs_state (1|2|3, checked), fsrs_step, fsrs_stability, fsrs_difficulty, last_reviewed_at`, plus `next_due_at`.

See `scheduling_algorithm.md` for what they mean.

Tables

`folders`: the notebook's folder tree. `id, parent_folder_id (nullable, self-referencing, cascade), name, created_at`

`notes`: belongs to a folder. `id, folder_id -> folders.id (cascade), title, body, source_filename (nullable), sort_order (default 0), created_at, updated_at`

`sort_order` makes bulk import's drag-to-reorder step stick: `NotesRepo.listByFolder()` orders by it, then by `title`.

`questions`: one row per generated (or edited) quiz question, each an FSRS card. `next_due_at` is `NOT NULL`: a new question is due at once. `id, note_id -> notes.id (cascade), question_type ('short_answer' | 'multiple_choice'), prompt, expected_answer, <FSRS columns>, next_due_at, created_at, active (0|1), is_manually_edited (0|1)`

`question_choices`: multiple-choice options only. `id, question_id -> questions.id (cascade), choice_text, is_correct (0|1), sort_order`

`question_history` : append-only log of every answer (`ReviewsRepo` has no update or delete). `id, question_id -> questions.id (cascade), asked_at, given_answer, is_correct (0|1), rating (1-4), grading_method ('exact_match' | 'keyword_match' | 'mcq_exact'), next_due_after, stability_after (nullable)`

`problems` : the LeetCode-style tracker. The FSRS columns and `next_due_at` stay NULL until the first attempt, so an untried problem is never due. Tags are one comma-separated, lowercased string. `id, title, url (nullable), platform, difficulty ('easy' | 'medium' | 'hard'), tags, solution, notes, <FSRS columns>, next_due_at (nullable), created_at, updated_at`

`problem_attempts` : append-only log of every attempt. `id, problem_id -> problems.id (cascade), attempted_at, solved (0|1), duration_minutes (nullable, >= 0), rating (1-4), notes, next_due_after, stability_after (nullable)`

A problem's status (to do / attempted / solved) isn't stored: `ProblemsRepo` derives it from the attempt count and the latest attempt's `solved` , in the same query.

Migration v1 to v2

v1 scheduled questions with `interval_index` (0-6) under a `CHECK (... BETWEEN 0 AND 6)` constraint, which SQLite can't drop on every Android version. So `questions` and `question_history` are rebuilt: rows are read, the tables dropped and recreated with the v2 DDL, and rows copied back **with their original ids**, each question converted with `cardFromLegacy()` . `question_choices` is untouched, and because foreign keys are off during the upgrade, dropping `questions` doesn't cascade into it. `problems` and `problem_attempts` are created empty. The migration test (`test/storage/db_test.dart`) builds a real v1 file from `schemaStatementsV1` and upgrades it through `openAppDatabase()` .

Design notes

- No ORM: plain `sqflite` and one repository class per table group, each taking a `Database` (dependency injection), so tests use real in-memory SQLite via `sqflite_common_ffi` .
- Indexes exist on every foreign key and on both `next_due_at` columns (the "what's due" queries).
- `ProblemsRepo.recordAttempt()` updates the card and inserts the attempt in one transaction.
- No many-to-many `question_source_notes` table: without note-linking, a question has exactly one source note. See `future_work.md` .